

ParaFormer: A Hybrid Graph Neural Network and Transformer Approach for Pre-Routing Parasitic RC Prediction

Jongho Yoon, Jakang Lee, Donggyu Kim, Junseok Hur, and Seokhyeong Kang

Department of Electrical Engineering, POSTECH

Pohang, Republic of Korea

shkang@postech.ac.kr

ABSTRACT

Predicting the quality of post-route design at an early stage can reduce overall design time. To achieve this, we propose ParaFormer, a pre-routing parasitic RC prediction framework. This framework integrates a heterogeneous graph neural network (HGNN) and a graph transformer to capture the topological and geometric information of circuit data. The HGNN model represents circuit data as heterogeneous graphs to learn complex topological relationships, while the graph transformer calculates attention between each net to learn geometric relationships. Our framework predicts parasitic RC, enabling RC tree modeling and SPEF file generation. This allows the predicted results to be utilized in timing and power analysis using commercial tools. Additionally, we incorporate gradient normalization to reduce the imbalance between different objectives in multi-task learning, improving overall model performance. Experimental results show that ParaFormer achieves R^2 scores of 0.9901 and 0.9630 for resistance and capacitance, respectively. In timing analysis, it achieves R^2 scores of 0.9749 for wire delay and 0.9876 for cell delay, with a MAPE of 1.45% in power analysis. These results indicate that our method is highly effective for timing and power prediction in the early design stage.

ACM Reference Format:

Jongho Yoon, Jakang Lee, Donggyu Kim, Junseok Hur, and Seokhyeong Kang. 2025. ParaFormer: A Hybrid Graph Neural Network and Transformer Approach for Pre-Routing Parasitic RC Prediction. In *30th Asia and South Pacific Design Automation Conference (ASPDAC '25)*, January 20–23, 2025, Tokyo, Japan. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3658617.3697599>

1 INTRODUCTION

The physical design flow often encounters unsatisfactory design quality after routing, necessitating modifications at early stages. Since routing is time-consuming, modifications requiring rerouting can significantly increase overall design time. Machine learning (ML)-based approaches have been studied to predict post-routing design quality early in the design process, enabling a single routing iteration and reducing turnaround time, as illustrated in Fig. 1.

Several studies [1, 2, 3, 4, 5, 6] have employed learning-based methods to predict path delay or slack at early stages. However, directly predicting delays requires a significant amount of slew data, as cell delays are determined by propagated slew data along the path. To overcome these challenges, Chhabria *et al.* [7] proposed

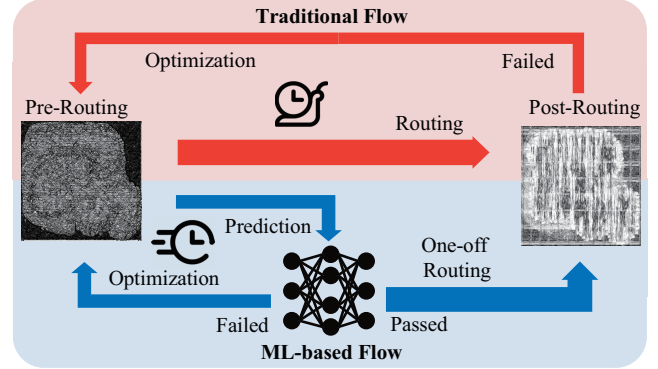


Figure 1: The ML-based physical design requires only a single routing, even through multiple optimization iterations, unlike the traditional flow.

a framework using XGBoost [8] to predict wire load capacitance. This approach allows the STA tool to analyze delays, mitigating the issues associated with directly predicting delays.

Nonetheless, this approach[7] struggles to handle the topological information of netlists and fails to consider geometric information such as instance density. Therefore, a method that accurately captures both elements is necessary to create an accurate and generalized prediction model. In this context, graph neural network (GNN) has garnered attention. GNN effectively captures connectivity features because it processes graphs as inputs. However, even with GNN, it is challenging to consider geometric information simultaneously. To address this limitation, methods that integrate both topological and geometric information have been explored. Yang *et al.* [9] proposed a method to predict routability by combining topological and geometric information. This approach uses circuit graph data to learn topological information and constructs a geometric graph by connecting adjacent cells to capture spatial relationships. However, this method also has limitations as it is challenging to use when target objects are not fixed.

In this paper, we introduce **ParaFormer**, a pre-routing parasitic prediction framework that utilizes a heterogeneous GNN (HGNN) and a graph transformer. Our method leverages heterogeneous graphs to capture the diverse topological characteristics of circuits. Additionally, we employ a transformer to calculate attention scores across all nets, enabling the learning of geometric relationships between them. Our model also implements multi-task learning to predict the resistance and capacitance of each net within the circuit. When the scale of losses for these tasks differs, it is crucial to balance learning to prevent biased training, which could degrade analysis performance. We address this by using gradient normalization (GradNorm) [10] to adjust the magnitude of the loss gradients for each task. Our framework accurately predicts wire parasitic RC, constructs an RC tree using the π -model, and extracts this data

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

ASPDAC '25, January 20–23, 2025, Tokyo, Japan

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0635-6/25/01.

<https://doi.org/10.1145/3658617.3697599>

into standard parasitic exchange format (SPEF) files. This allows external tools to perform early stage post-route timing and power analysis based on our predictions. The main contributions of this paper are as follows:

- We introduce **ParaFormer**, combining HGNN and graph transformer for pre-routing parasitic prediction.
- Our method captures both topological and geometric information of circuits, utilizing the heterogeneous graph to learn the intricate relationships and the transformer to evaluate attention scores across all net pairs.
- We implement GradNorm to tailor learning rates for each task. The results of our ablation study confirm that these techniques effectively improve downstream task performance.
- The framework enables the construction of RC trees using the π -model and the extraction of this data into SPEF files. It facilitates early stage post-route timing and power analysis with external tools, achieving R^2 scores of 0.9749 and 0.9876 for net and cell delays, and a 1.45% mean absolute percentage error (MAPE) for power.

The rest of this paper is organized as follows: Section 2 introduces the background on parasitic prediction and heterogeneous graph neural networks. Section 3 describes the proposed parasitic prediction model. Section 4 presents experimental results and analysis. Finally, Section 5 concludes the paper.

2 PRELIMINARIES

2.1 Pre-Routing Timing Prediction

Recent research in VLSI design has utilized various ML techniques to enhance timing prediction accuracy in the early stages of physical design. Barboza *et al.* [1] first introduced ML for pre-routing timing prediction to reduce pessimism. However, these ML-based methods often need to properly learn information that impacts timing metrics, such as the connectivity of the circuit. In response, He *et al.* [3] proposed a slack prediction model based on GNN inspired by timing engine structures, introducing a delay propagation model that addresses the limited receptive field issue and resolves the over-smoothing problem. He *et al.* [6] leveraged heterogeneous graph learning to enable more precise node embedding and extend the receptive field for timing paths through a transformer model. However, these timing prediction methods [1, 3, 6] require substantial learning data. Particularly in the non-linear delay model (NLDM), variables such as input slew, output capacitance, and input combinations must be considered to enhance the accuracy of timing predictions. In this model, gate delay and output slew are determined by three main factors: input slew, output capacitance, and the input vector. These factors are crucial because the output slew from one gate becomes the input slew for the next gate in the path. Therefore, even minor errors in predicting the initial slew can propagate and accumulate, leading to significant inaccuracies in the overall path delay prediction. A thorough understanding of the characteristics of slew propagation and securing sufficient data on the factors that determine gate delays are essential for accurate timing prediction. Predicting parasitic RC, determined solely by the physical properties of the wires, allows for effective learning with less data. Parasitic RC prediction is closely linked to timing prediction and is not affected by path-based features like slew propagation, making it advantageous for various timing analyses.

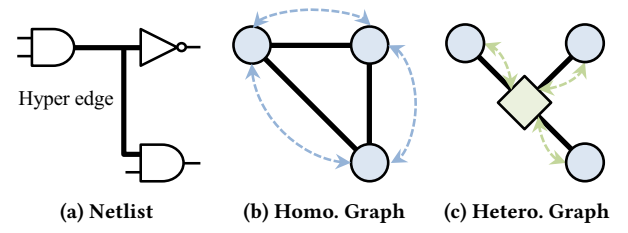


Figure 2: Heterogeneous graphs effectively handle hyper-edges such as nets in a circuit.

2.2 Graph Neural Network for Circuit

GNN is an artificial neural network designed to work with graph data structures. Graphs consist of nodes and edges, representing a wide range of real-world data. GNN captures the relationships and interactions between nodes in a graph using message passing. Through this technique, nodes aggregate information from neighboring nodes to update their states. Recently, several studies have utilized circuits as graph data to leverage the features of GNN. Specifically, they aimed to perform various prediction tasks using netlists, representing the connectivity information of circuit instances, at an early stage [4, 6]. However, there are limitations to modeling circuits with graphs consisting of only one type of nodes and edges. The loss of information from simplifying the diverse components and complex connectivity within circuits can lead to task performance degradation. Modeling circuits as heterogeneous graphs allows for capturing the actual structural characteristics of circuits more effectively by including various types of nodes and edges between them. As shown in Fig. 2, heterogeneous graphs can handle hyper-edges, while homogeneous graphs cannot. Hyper-edges, a characteristic feature of netlists, allow GNN to aggregate information between gates and share features more effectively. This results in a more accurate representation of circuit connectivity and significantly enhances learning efficiency.

HGNN [12] has been proposed to handle the unique structure of heterogeneous graphs. In HGNN, the message passing has been customized to accommodate the diverse node and edge types present within the graph. Each node in an HGNN aggregates features from its neighboring nodes based on the specific type of edges between them. The aggregated features from different edge types are then combined to update the feature vector. Specifically, the feature update for a node u can be represented as follows:

$$\mathbf{h}_u^{(l+1)} = \sigma \left(\sum_{e \in \mathcal{E}} \text{AGGREGATE}_e \left(\sum_{v \in \mathcal{N}_e(u)} \mathbf{W}_e^{(l)} \cdot \mathbf{h}_v^{(l)} \right) \right), \quad (1)$$

where $\mathbf{h}_u^{(l)}$ is the feature vector of node u at layer l , σ is a non-linear activation function, $\mathcal{E}$ is the set of all edge types, $\mathcal{N}_e(u)$ represents the set of neighbors of node u connected through edge e , $\mathbf{W}_e^{(l)}$ is the learnable weight matrix of edge e at layer l , and AGGREGATE_e is the aggregate function combining the features from edge type e .

Considering the physical nature of circuits, it is crucial to integrate both topological and geometrical information in graph data for improved prediction performance. Yang *et al.* [9] introduced CircuitGNN, a versatile multi-stage GNN that uses a heterogeneous graph to integrate topological and geometrical information from both the netlist and design layout stages. This model constructs a geometric graph by linking adjacent cells and processes this information using GNN, effectively capturing the spatial characteristics

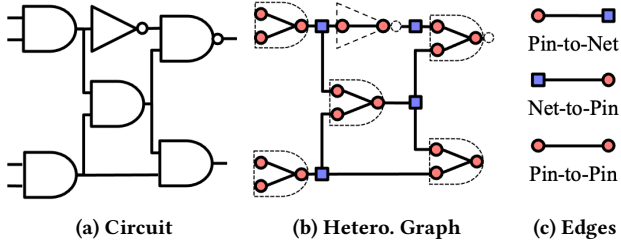


Figure 3: Convert (a) the circuit design into (b) a heterogeneous graph. (c) The circuit graph has three types of edges.

Table 1: Pin and Net Node Features

Type	Features	Size
Pin	coordinate of pin (x, y)	2
	early/late and rise/fall capacitance of pin	4
Net	coordinate of net bounding box (lx, ly, ux, uy)	4
	area of net bounding box	1
	net degree	1
	net res, cap, and wirelength	3

of the circuit. Similarly, Zhao *et al.* [11] proposed HybridNet, a dual-branch network to capture both geometrical interactions and topological relationships in the netlist. However, it is essential to note that these methods, which involve creating graphs by linking adjacent cells, are inadequate for tasks targeting nets, such as parasitic RC prediction.

3 PROPOSED METHOD

In this section, we propose a parasitic RC prediction framework based on HGNN and graph transformer. Section 3.1 demonstrates how to represent circuit netlists as heterogeneous graphs. Section 3.2 details an HGNN-based parasitic prediction model that includes a graph transformer to consider the geometric information of the circuit. Finally, Section 3.3 demonstrates the RC tree construction and SPEF file extraction to utilize parasitic values in external tools.

3.1 Netlist Graph Representation

We aim to predict the parasitic RC of circuits at the pre-routing step. To improve the accuracy of our predictions, it is crucial to represent the circuits effectively. Therefore, we introduce a heterogeneous graph model that consists of two types of nodes: pins and nets, as depicted in Fig. 3. The features of each node are described in detail in Table 1. Heterogeneous graphs can capture the diverse interactions and complex structures within circuits. For instance, pins and nets represent different entities within a circuit, each with unique attributes essential for accurate parasitic RC prediction. We employ three types of edges to capture the intricate relationships and characteristics within the circuit: *pin-to-pin*, *net-to-pin*, and *pin-to-net* connections. Within a single cell, the connections between input and output pins are defined as *pin-to-pin*. *net-to-pin* edges represent the connections from a net to the pins it connects, capturing the interaction where multiple pins might share the same net. *pin-to-net* edges define the reverse relationship, from a pin to the nets it connects to, ensuring that the model understands both directions of interaction. These edge types reflect real-world circuit connections, allowing the model to accurately represent the flow of signals and interactions within the circuit. The parasitic RC values we aim to predict are labels of the net nodes.

3.2 ParaFormer

In this section, we detail our method, which integrates heterogeneous GNN (HGNN) and a graph transformer to predict the parasitic RC values in a circuit netlist. This approach leverages both topological and geometric information for enhanced prediction accuracy. An overview of ParaFormer is illustrated in Fig. 4.

Heterogeneous GNN HGNN is designed to handle the heterogeneous structure of the circuit graph by utilizing multiple layers. Within each layer, different kernels are defined for each edge type, and these kernels perform message passing tailored to the specific properties of the corresponding edge types. This allows the network to effectively learn the unique characteristics and interactions of pins and nets. For net nodes, a single node embedding is obtained as they are connected to neighboring nodes through only *pin-to-net* edges. However, pin nodes are connected through both *net-to-pin* and *pin-to-pin* edges. Therefore, for pin nodes, the node embeddings generated from each edge type are averaged to obtain the final pin node embedding.

Graph Transformer While HGNN effectively captures the topological information of the circuit, it does not fully account for the geometric relationships. Previous works [9, 11] have proposed frameworks that consider the geometric information of netlists by defining geometric graphs for cells. However, these methods, which involve creating graphs by linking adjacent cells, are ineffective for tasks targeting nets. To address this, we integrate a graph transformer into our model, which enhances HGNN by learning the geometric information. The transformer employs an attention mechanism that calculates attention scores between all pairs of net nodes. This mechanism allows the model to capture the geometric relationships between the nets by learning the importance of each net pair. However, computing attention scores for all net pairs results in a computational complexity of $O(n^2)$, making it challenging to process large circuit graphs. To mitigate this issue, we replace the softmax function used for attention score computation with a kernel function [13, 14]. In a traditional Transformer, attention score calculations are computed as follows:

$$z_u^{(l+1)} = \sum_{v=1}^N \frac{\exp\left((Q_u^{(l)})^\top (K_v^{(l)})\right)}{\sum_{w=1}^N \exp\left((Q_u^{(l)})^\top (K_w^{(l)})\right)} \cdot V_v^{(l)}, \quad (2)$$

where $Q_u^{(l)}$ is the query matrix of node u at layer l , $K_v^{(l)}$ is the key matrix of node v at layer l , $V_v^{(l)}$ is the value matrix of node u at layer l . However, a kernel function can replace the softmax operation to reduce computational complexity as follows:

$$z_u^{(l+1)} = \sum_{v=1}^N \frac{\phi(Q_u^{(l)})^\top \phi(K_v^{(l)})}{\sum_{w=1}^N \phi(Q_u^{(l)})^\top \phi(K_w^{(l)})} \cdot V_v^{(l)}, \quad (3)$$

where ϕ represents the kernel function to approximate softmax operation. This approach can be further simplified as follows:

$$z_u^{(l+1)} = \frac{\phi(Q_u^{(l)})^\top \sum_{v=1}^N \phi(K_v^{(l)}) \cdot (V_v^{(l)})^\top}{\sum_{w=1}^N \phi(Q_u^{(l)})^\top \phi(K_w^{(l)})}. \quad (4)$$

The summations in the numerator and denominator can be computed once and reused. Therefore, this kernel-based approach approximates the attention computation and scales linearly with the number of nodes, $O(n)$, compared to $O(n^2)$ of the traditional softmax-based computation.

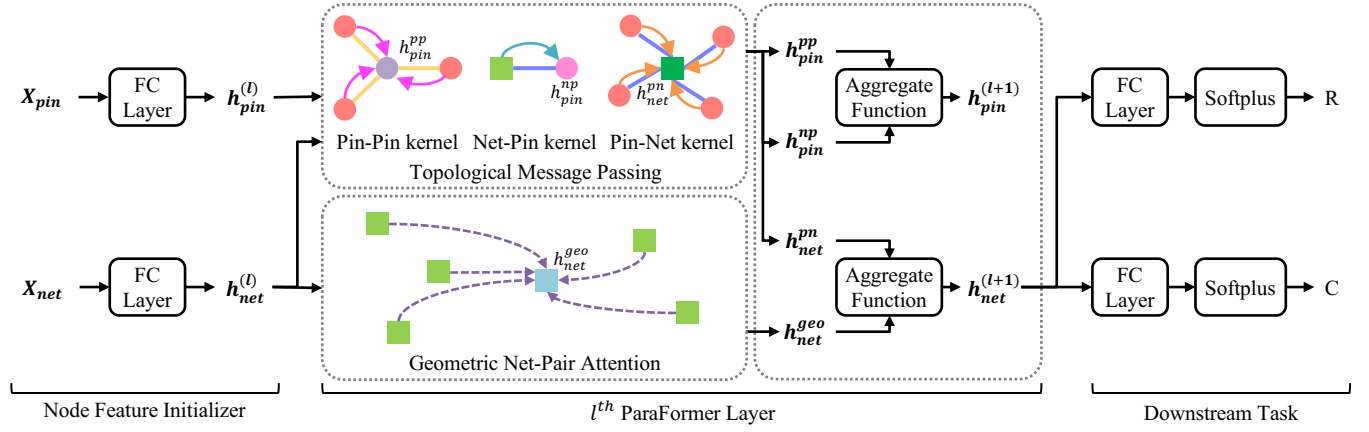


Figure 4: Overview of ParaFormer. The message passing of HGNN leverage topological information, while the attention mechanism of the Transformer leverages geometric information.

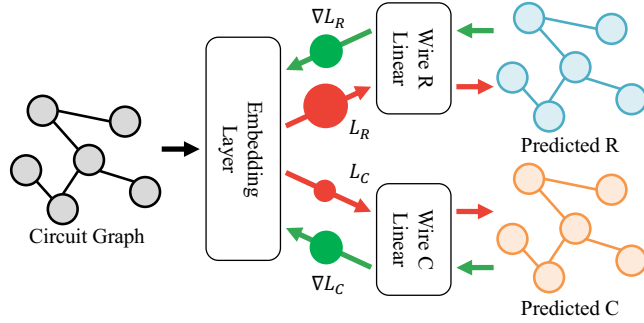


Figure 5: Operation of GradNorm. GradNorm adjusts gradients (∇L_R , ∇L_C) to balance learning when there is an imbalance between the loss for R (L_R) and the loss for C (L_C).

ParaFormer We combine HGNN and graph transformer to leverage topological and geometric information. HGNN processes the heterogeneous graph to capture topological information, while the graph transformer computes attention scores to learn geometric relationships between nets. The embeddings from HGNN and graph transformer are combined using an aggregation function, the average function in this work, to leverage the complementary strengths of both types of information. The aggregated embeddings are then used to predict the parasitic resistance (R) and capacitance (C) values. The aggregated embeddings are passed through a fully connected (FC) layer, for each RC prediction. Each predicted value is ensured to be non-negative by passing them through the *Softplus* function.

Task Balancing ParaFormer performs multi-task learning to predict parasitic R and C values. However, each task has different loss scales, which can negatively affect the overall model performance by causing varying training speeds. To address this issue, we apply the GradNorm technique [10], as shown in Fig. 5. GradNorm adjusts the magnitude of the gradients for the loss of each task, ensuring that all tasks learn at similar rates. The weights are adjusted based on the gradient magnitudes of each task as follows:

$$w_i^{(t+1)} = w_i^{(t)} \cdot \left(\frac{\|G_i\|}{\frac{1}{T} \sum_j \|G_j\|} \right)^\gamma, \quad (5)$$

where $w_i^{(t)}$ represents the weights at training time t for task i , T is the total number of tasks, and G_i is the gradient vector for the loss

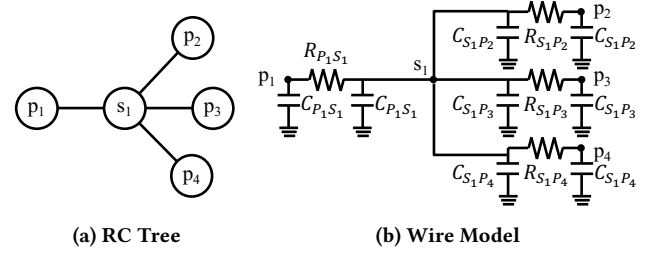


Figure 6: (a) RC tree construction and (b) wire modeling methods for multi-pin wires.

of a task i . The parameter γ is an adjustment parameter, and the larger its value, the more strongly it modulates the differences in gradient magnitudes between tasks. GradNorm dynamically adjusts the learning rate by modulating the weights: reducing it for tasks with rapidly decreasing loss and increasing it for tasks with slowly decreasing loss. The updated weights are then combined with each loss to calculate the total loss.

3.3 RC Tree Modeling

ParaFormer predicts the parasitic resistance (R) and capacitance (C) values for all nets. Modeling the RC tree is necessary to utilize these predicted values in a commercial tool. Initially, we perform RC tree construction as shown in Fig. 6(a). Each net consists of one driver pin and multiple sink pins, and we connect them by adding a virtual Steiner point located at the center of the net bounding box. The R and C values for the segments are relatively assigned based on the Manhattan distance between the pin and the Steiner point. The R and C values for the segment s connecting pin i to the Steiner point p can be calculated as follows:

$$R_s = \frac{|x_i - x_p| + |y_i - y_p|}{\sum_{j \in \mathcal{P}} |x_j - x_p| + |y_j - y_p|} \cdot R_{pred}, \quad (6)$$

$$C_s = \frac{|x_i - x_p| + |y_i - y_p|}{\sum_{j \in \mathcal{P}} |x_j - x_p| + |y_j - y_p|} \cdot C_{pred}, \quad (7)$$

where R_{pred} and C_{pred} are the predicted R and C values, x_i and y_i are coordinates of the pin i , and $\mathcal{P}$ is the set of all pins. The calculated R values are assigned to the segment, and the C value is split evenly between the two connected pins. Thus, as shown in Fig. 6(b), we use the Π -model to model the wire. The generated wire model can be extracted into a SPEF file for use in commercial tools.

Table 2: Comparison of Parasitic RC Prediction (R^2 score) and Runtime (sec.) Results. The best test results of prediction are marked in **blue, and the second-best are underlined.**

Design	Wire R					Wire C					Runtime					
	ML	GNN	HGNN	Ours	Ours-G ¹	ML	GNN	HGNN	Ours	Ours-G ¹	Trad. ²	ML	GNN	HGNN	Ours	Ours-G ¹
D1	0.9886	0.9563	0.9581	0.9788	0.9621	0.9845	0.8760	0.9520	0.9784	0.9795	37	0.001	0.002	0.004	0.007	0.007
D2	0.9758	0.9716	0.9653	0.9826	0.9780	0.9766	0.8888	0.9326	0.9747	0.9852	134	0.002	0.002	0.005	0.008	0.008
D3	0.9620	0.9531	0.9478	0.9639	0.9488	0.9689	0.8754	0.9412	0.9430	0.9647	280	0.002	0.002	0.005	0.009	0.009
D4	0.9820	0.9765	0.9837	0.9828	0.9780	0.9736	0.9015	0.9513	0.9668	0.9755	257	0.003	0.003	0.006	0.012	0.012
D5	0.9896	0.9776	0.9638	0.9872	0.9885	0.9901	0.9579	0.9049	0.9417	0.9921	710	0.003	0.003	0.006	0.011	0.011
D6	0.9961	0.9786	0.9799	0.9936	0.9896	0.9930	0.9724	0.9858	0.9885	0.9900	764	0.012	0.004	0.010	0.021	0.021
D7	0.9925	0.9709	0.9644	0.9866	0.9777	0.9932	0.9642	0.9672	0.9909	0.9958	3033	0.015	0.004	0.011	0.025	0.025
D8	0.9984	0.9801	0.9880	0.9948	0.9812	0.9958	0.9138	0.9498	0.9529	0.9832	893	0.018	0.005	0.013	0.032	0.032
D9	0.9996	0.9988	0.9978	0.9979	0.9990	0.9996	0.9924	0.9938	0.9939	0.9926	10171	0.058	0.015	0.052	0.148	0.148
D10	0.9794	0.9869	0.9797	0.9904	0.9869	0.9420	0.9282	0.9722	0.9871	0.9915	263	0.002	0.002	0.006	0.009	0.009
D11	0.9431	0.9597	0.9521	0.9812	0.9633	0.9278	0.8227	0.8384	0.9318	0.9941	12936	0.004	0.003	0.007	0.015	0.015
D12	0.9214	0.9677	0.9669	0.9954	0.9741	0.9618	0.9455	0.9707	0.9677	0.9969	1238	0.016	0.005	0.013	0.030	0.030
D13	0.6056	0.9769	0.9880	0.9934	0.9895	0.7137	0.9355	0.9601	0.9656	0.9842	4889	0.033	0.008	0.026	0.081	0.081
Train avg.	0.9872	0.9737	0.9721	0.9854	0.9781	0.9861	0.9269	0.9532	0.9701	0.9843	1809	0.0126	0.0044	0.0125	0.0303	0.0303
Test avg.	0.8624	0.9728	0.9717	0.9901	0.9785	0.8863	0.9080	0.9354	0.9630	0.9917	4831	0.0139	0.0046	0.0127	0.0338	0.0338

¹with GradNorm, ²Traditional routing flow**Table 3: Design statistics for parasitic prediction.**

Index	Design	# Cells	# Nets	# Pins
D1	<i>spi_top</i>	1829	1771	4628
D2	<i>usbf_top</i>	7764	7896	17759
D3	<i>aes_cipher_top</i>	9366	9237	26526
D4	<i>fpu</i>	17206	19362	36833
D5	<i>wb_conmax_top</i>	18040	16624	46984
D6	<i>eth_top</i>	33191	33270	98527
D7	<i>ldpc_decoder_802_3an</i>	43289	41240	96465
D8	<i>des3</i>	59428	59364	128221
D9	<i>mpeg2_top</i>	376811	384545	1101167
D10	<i>pci_bridge32</i>	10661	10503	27143
D11	<i>keccak</i>	25771	25257	55027
D12	<i>vga_enh_top</i>	55145	55060	132504
D13	<i>tate_pairing</i>	167474	166309	489825

4 EXPERIMENTAL RESULTS

4.1 Experimental Setup

The experiments were conducted on a server with an *Intel Xeon Gold 6226R* CPU and an *NVIDIA TITAN RTX* GPU. We utilized open-source benchmarks provided by *OpenCores* [18] to train and evaluate the parasitic prediction model, as detailed in Table 3. The dataset comprises nine training designs and four testing designs. We used *Synopsys Design Compiler* [19] for synthesis and *Cadence Innovus* [20] for placement and routing with an *ASAP7* PDK [15]. For each design, we varied the options to generate 20 different P&R results, which were used as augmented data. The parasitic RC prediction model was implemented using *PyTorch* [16] and *DGL* [17]. All models used in the experiments consist of 2 layers, with the hidden state feature size set to 16. During training, the learning rate γ was set to 0.001, learning rate decay $\Delta\gamma$ to 0.1 per 20 epochs, weight decay η to 0.001, and training epoch e to 100.

4.2 Parasitic RC Prediction

For the parasitic RC prediction task, we compared ParaFormer against traditional ML approaches such as *XGBoost* [8] and typical GNN and HGNN models, serving as baselines. Additionally, we experimented with a ParaFormer applying GradNorm to enhance

the performance of our model. In these cases, ML was trained solely on features from net nodes, and GNN was trained on graph data consisting only of net nodes. The R^2 score results and runtime of each method are presented in Table 2.

The ML model achieved high R^2 scores for both wire R and C on the training dataset but displayed significantly lower performance on the test dataset than other graph-learning-based models. This highlights the limitations of ML models that only consider net features without learning the structural properties of the graph, resulting in poor generalization. The GNN model showed lower accuracy for C than the HGNN model due to its reliance on a homogeneous graph, which misses crucial information necessary for accurate parasitic prediction. Consequently, the GNN may struggle to effectively learn specific patterns or structures within circuit graphs. HGNN model performed better, especially for C, than the GNN but still lagged behind ParaFormer in terms of accuracy.

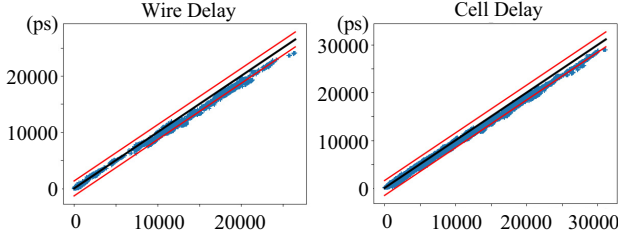
ParaFormer (Ours), leveraging both topological and geometric information through the integration of the graph transformer, achieved significant improvements in prediction accuracy. The graph transformer component of ParaFormer effectively learns the geometric relationships between nets, enhancing the model’s ability to predict parasitic RC with high accuracy. This is evident in the results where ParaFormer outperformed the baselines for both wire R and C across multiple designs. Specifically, ParaFormer achieved the R^2 score of 0.9901 for wire R and 0.9630 for wire C prediction.

ParaFormer with GradNorm (Ours-G) further reduced learning imbalances between R and C predictions, resulting in more consistent performance. ParaFormer without GradNorm showed a high R^2 score for wire R but a lower score for C, indicating an imbalance in learning the two tasks. Remarkably, the loss scale for R was observed to be larger than that for C, causing the model’s learning to be biased towards R. However, with GradNorm, the model achieved more balanced and superior performance in both tasks. Specifically, Ours-G maintained a high R^2 score of 0.9785 for wire R prediction while significantly improving the R^2 score for C to 0.9917, indicating that the learning for C was also well achieved. Additionally, ParaFormer requires less than an hour to train using a single GPU, and the time taken to infer the parasitics of a single circuit is very short, typically less than 0.1 seconds.

Table 4: Comparison on Net and Cell Delay Analysis (R^2 score) and Power Analysis (MAPE, %) Results. The best results are marked in blue.

Design	Wire Delay					Cell Delay					Power				
	ML	GNN	HGNN	Ours	Ours-G*	ML	GNN	HGNN	Ours	Ours-G*	ML	GNN	HGNN	Ours	Ours-G*
D1	0.9431	0.8179	0.8787	0.9173	0.9340	0.9901	0.9424	0.9738	0.9793	0.9888	2.080	0.870	3.050	1.220	1.680
D2	0.8821	0.8512	0.8538	0.8837	0.9156	0.9775	0.9428	0.9442	0.9811	0.9893	0.500	3.990	2.080	3.420	0.460
D3	0.9442	0.7835	0.8808	0.9143	0.9413	0.9853	0.9611	0.9763	0.9678	0.9845	0.960	1.700	2.100	5.970	0.990
D4	0.9565	0.8314	0.9346	0.9214	0.9511	0.9907	0.9917	0.9822	0.9815	0.9816	4.110	8.570	7.010	6.840	5.490
D5	0.9514	0.9157	0.8752	0.9110	0.9604	0.9866	0.9302	0.9212	0.9496	0.9907	6.330	19.210	10.600	14.780	4.190
D6	0.9858	0.9371	0.9623	0.9812	0.9772	0.9975	0.9300	0.9739	0.9897	0.9907	0.360	1.010	0.890	1.660	0.480
D7	0.9029	0.5510	0.8572	0.9371	0.9368	0.9744	0.9404	0.9395	0.9750	0.9777	4.700	12.920	16.700	8.260	9.260
D8	0.7950	0.6406	0.8256	0.8890	0.9420	0.9664	0.9372	0.9689	0.9808	0.9919	0.980	7.940	2.720	13.520	0.420
D9	0.9747	0.8612	0.9116	0.9512	0.9850	0.9898	0.8809	0.9116	0.9599	0.9935	0.100	1.210	0.820	0.740	0.080
D10	0.8790	0.7239	0.9084	0.8657	0.9587	0.9693	0.9492	0.9740	0.9801	0.9874	0.290	0.690	0.800	0.340	0.430
D11	0.8741	0.9189	0.9243	0.9660	0.9915	0.8609	0.9104	0.9179	0.9637	0.9899	11.300	8.640	9.680	5.620	2.500
D12	0.9273	0.8427	0.9124	0.9305	0.9714	0.9553	0.8649	0.9290	0.9401	0.9928	5.130	4.330	0.880	3.210	0.970
D13	0.7246	0.8184	0.8503	0.8670	0.9780	0.7242	0.8130	0.8374	0.8543	0.9803	1.590	4.120	2.540	4.070	1.910
Train avg.	0.9262	0.7988	0.8866	0.9229	0.9493	0.9843	0.9396	0.9546	0.9739	0.9876	2.2356	6.3800	5.1078	6.2678	2.5611
Test avg.	0.8513	0.8260	0.8989	0.9073	0.9749	0.8774	0.8844	0.9146	0.9346	0.9876	4.5775	4.4450	3.4750	3.3100	1.4525

*with GradNorm

**Figure 7: Wire and cell delay analysis for one sample of D12. The red line represents the boundary line for $\pm 5\%$ error.**

4.3 Timing & Power Analysis

We extracted SPEF files through wire modeling using the predicted parasitic RC and analyzed the timing and power using *Synopsys PrimeTime* [21]. We evaluated the timing and power analysis results using the predicted parasitic from each model and compared them to the post-route results. These results are presented in Table 4.

The results clearly show that our proposed ParaFormer outperforms traditional ML methods and typical GNN in terms of timing and power analysis accuracy. ParaFormer, leveraging both topological and geometric information through the integration of a graph transformer, showed significant improvements. Specifically, ParaFormer achieved high R^2 scores for both wire delay and cell delay and relatively low mean absolute percentage error (MAPE) for power estimation.

ParaFormer with GradNorm further enhanced the performance, achieving the best results among all models. Ours-G maintained a high R^2 score for wire delay while significantly improving the R^2 score for cell delay and achieving the lowest MAPE for power analysis. This indicates that GradNorm helped balance the learning between R and C predictions, leading to more accurate downstream tasks such as timing and power analysis. Specifically, Ours-G achieved an R^2 score of 0.9749 for wire delay and 0.9876 for cell delay, with a MAPE of 1.45% for power estimation, showcasing its precision and reliability. The superior performance of Ours-G can be attributed to its balanced learning approach, which ensures that both resistance and capacitance predictions are equally accurate.

This balance is crucial for downstream tasks like timing and power analysis, as both resistance and capacitance play significant roles in determining these characteristics. By mitigating the learning imbalance, Ours-G was able to provide highly accurate predictions that closely match the post-route results. Fig. 7 illustrates the prediction results of wire and cell delay by ParaFormer for a sample in the D12 design.

5 CONCLUSION

In this study, we propose a parasitic RC prediction framework for early design stage timing and power analysis. Our model combines HGNN and an all-pair graph transformer to consider both topological and physical information of circuit elements, utilizing a kernelized softmax function for scalability. Our method offers several contributions: Firstly, we showed that representing the circuit netlist as a heterogeneous graph captures various circuit characteristics, improving prediction accuracy. Second, we demonstrated that ParaFormer outperforms traditional ML and homogeneous graph-based GNN. Thirdly, our approach enables RC tree modeling using predicted parasitic RC values and generating SPEF files for post-route timing and power analysis with commercial tools. Lastly, we enhanced model performance with GradNorm to balance learning between parasitic R and C predictions. Experimental results showed that ParaFormer achieved R^2 scores of 0.9901 for R and 0.9630 for C. In timing analysis, it achieved R^2 scores of 0.9749 for wire delay and 0.9876 for cell delay, with a MAPE of 1.45% in power analysis. These results indicate that ParaFormer is highly effective for early-stage timing and power analysis.

ACKNOWLEDGMENTS

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2023-00222609, Development of thermal modelling and heat-spreading architecture for heterogeneous integrating package of AI chiplets), Samsung Electronics Co., Ltd. And, the EDA tool was supported by the IC Design Education Center (IDEC), Korea.

REFERENCES

- [1] E. C. Barboza, N. Shukla, Y. Chen, and J. Hu, "Machine Learning-Based Pre-Routing Timing Prediction with Reduced Pessimism", *Proc. DAC*, 2019
- [2] T. Yang, G. He, and P. Cao, "Pre-routing Path Delay Estimation Based on Transformer and Residual Framework", *Proc. ASP-DAC*, 2022
- [3] X. He, Z. Fu, Y. Wang, C. Liu, and Y. Guo, "Accurate Timing Prediction at Placement Stage with Look-Ahead RC Network", *Proc. DAC*, 2022
- [4] Z. Guo, M. Liu, J. Gu, S. Zhang, D. Z. Pan, and Y. Lin, "A Timing Engine Inspired Graph Neural Network Model for Pre-Routing Slack Prediction", *Proc. DAC*, 2022
- [5] P. Cao, G. He and T. Yang, "TF-Predictor: Transformer-based Pre-Routing Path Delay Prediction Framework", *IEEE TCAD* 42(7) 2023, pp. 2227-2237
- [6] G. He, W. Ding, Y. Ye, X. Cheng, Q. Song, and P. Cao, "An Optimization-aware Pre-Routing Timing Prediction Framework Based on Heterogeneous Graph Learning", *Proc. ASP-DAC*, 2024
- [7] V. A. Chhabria, W. Jiang, A. B. Kahng, and S. S. Sapatnekar, "A Machine Learning Approach to Improving Timing Consistency between Global Route and Detailed Route", *ACM TODAES* 29(1), 2023, pp. 1-25
- [8] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System", *Proc. KDD*, 2016
- [9] S. Yang, Z. Yang, D. Li, Y. Zhang, Z. Zhang, G. Song, and J. HAO, "Versatile Multi-stage Graph Neural Network for Circuit Representation", *Proc. NeurIPS*, 2022
- [10] Z. Chen, V. Badrinarayanan, C.-Y. Lee, and A. Rabinovich, "GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks", *Proc. ICML*, 2018
- [11] Y. Zhao, Z. Chai, Y. Lin, R. Wang, and R. Huang, "HybridNet: Dual-Branch Fusion of Geometrical and Topological Views for VLSI Congestion Prediction (Extended Abstract)", *Proc. NeurIPS*, 2022
- [12] C. Zhang, D. Song, C. Huang, A. Swami, and N. V. Chawla, "Heterogeneous Graph Neural Network", *Proc. KDD*, 2019
- [13] K. Choromanski, V. Likhoshesterov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser, D. Belanger, L. Colwell, and A. Weller, "Rethinking Attention with Performers", *Proc. ICLR*, 2021
- [14] Q. Wu, W. Zhao, Z. Li, D. Wipf, and J. Yan, "NodeFormer: A Scalable Graph Structure Learning Transformer for Node Classification", *Proc. NeurIPS*, 2022
- [15] L.T. Clark, V. Vashishtha, L. Shifren, A. Gujja, S. Sinha, B. Cline, C. Ramamurthy, and G. Yeric, "ASAP7: A 7-nm finFET predictive process design kit", *Microelectronics Journal*, 2016
- [16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimselshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An Imperative Style, High-Performance Deep Learning Library", *Proc. NeurIPS*, vol. 32, 2019, pp. 1-12
- [17] M. Wang, D. Zheng, Z. Ye, Q. Gan, M. Li, X. Song, J. Zhou, C. Ma, L. Yu, Y. Gai, T. Xiao, T. He, G. Karypis, J. Li, and Z. Zhang, "Deep Graph Library: A Graph-Centric, Highly-Performant Package for Graph Neural Networks", *arXiv preprint arXiv:1909.01315*, 2019
- [18] *Opencores*, <https://opencores.org>
- [19] *Synopsys Design Compiler v18.06 User Guide*
- [20] *Cadence Innovus Implementation System v21.16 User Guide*
- [21] *Synopsys PrimTime Static Timing Analysis v18.06 User Guide*